Speech Command Recognition for Hands-Free Music Control using Spectrogram-Based CNN with Reinforcement Learning Threshold Optimization

Almazor, Keith Ryan N.
School of Computing
Holy Angel University
Angeles City, Pampanga
+63 935 851 0718
keithryanalmanzor@gmail.com

Gonzales, John Reshley P.
School of Computing
Holy Angel University
Magalang, Pampanga
+63 992 724 3325
reshleygonzales11@gmail.com

Valderrama, Mark Harold T.
School of Computing
Holy Angel University
Angeles City, Pampanga
+63 968 600 1741
markvalderrama01@gmail.com

Yumul, Randel Angelo L.
School of Computing
Holy Angel University
Mabalacat City, Pampanga
+63 969 069 1662
randel.angelo10@gmail.com

ABSTRACT

Speech command recognition enables users to control devices using voice commands without physical interaction. This capability can improve convenience when users are performing other activities such as studying, working, or multitasking. However, background noise and unintended speech may lead to incorrect command activation in voice-controlled systems.

This study developed a speech command recognition system for hands-free music control using spectrogram-based convolutional neural networks (CNN). The system classifies simple voice commands including play, play with a song name, pause, next, previous, and stop from audio inputs. To reduce incorrect command activation, a reinforcement learning (RL) agent is used to optimize the decision threshold under asymmetric error conditions. The system was evaluated using accuracy, macro-F1 score, and expected cost. Ethical considerations such as accessibility, fairness across accents, and consent in audio data usage are also discussed.

CCS Concepts

• **Computing methodologies → Speech recognition**
• **Computing methodologies → Neural networks**
• **Computing methodologies → Reinforcement learning**
• **Information systems → Multimedia information systems**

Keywords

Speech command recognition; audio classification; spectrogram; convolutional neural network; reinforcement learning; voice control systems

1. INTRODUCTION

Music streaming and audio playback are widely used by students and professionals during study or work sessions. In many situations, users may want to control music playback without manually interacting with their device. Voice command recognition provides a convenient solution by allowing users to control music playback using simple spoken commands.

However, voice command systems may mistakenly activate commands due to background noise, conversation, or other audio sources. These errors can interrupt user activities and reduce system reliability. To address this problem, machine learning models such as convolutional neural networks (CNN) were used to classify audio commands from spectrogram representations.

Spectrograms convert audio signals into visual representations of frequency over time, making them suitable for deep learning models. CNN architectures have been widely used in audio classification tasks because they can automatically extract relevant features from spectrogram images.

Additionally, reinforcement learning (RL) can be applied to dynamically optimize the decision threshold of the classifier. Instead of using a fixed confidence threshold, an RL agent can learn the optimal threshold that minimizes costly errors such as false command activations or missed commands.

This project developed a speech command recognition system capable of recognizing simple commands such as play, play with a song name, stop, next, previous, and stop for music playback control while minimizing incorrect command detection through reinforcement learning threshold tuning. The developed system focuses on a lightweight and efficient architecture that can potentially be deployed on consumer devices for hands-free music control. The success of this system was measured against initial target thresholds of a Macro-F1 score of >70% and an Expected Cost (EC) below 0.5. The pipeline was designed to adhere to a strict 90-minute training runtime limit. To accommodate varied hardware availability, the training and evaluation scripts were engineered to automatically detect and utilize GPU acceleration (CUDA) if available, while maintaining full fallback compatibility with standard CPU execution.

2. RELATED WORK

2.1. CNN-Based Speech Recognition

Recent advancements in automatic speech recognition (ASR) have consistently highlighted the efficacy of deep learning architectures for acoustic processing. Hakan Ilgaz et al. (2024)

conducted a comprehensive comparative study on modern ASR systems, establishing convolutional neural networks (CNNs) as a robust, state-of-the-art baseline. Their findings demonstrate that CNNs are highly effective at capturing frequency invariants in speech, making them an ideal foundational architecture for audio classification.

Building upon this, Salah Mahdi et al. (2023) developed a microphone-based audio command classifier that successfully utilized a CNN to manage a limited vocabulary of 12 classes. Their research emphasizes the suitability and low-latency performance of CNNs for near-field, small-vocabulary applications, directly supporting the proposed system's architecture for processing a targeted five-command control set in real time.

2.2. Time–Frequency Audio Representation for Neural Networks

To effectively route audio inputs through a CNN, raw sound waves must be transformed into a compatible format. Rohan V. Sharan et al. (2020) validated the methodology of converting one-dimensional audio signals into two-dimensional, biologically inspired time-frequency representations—such as cochleagrams—prior to neural network classification.

This approach strongly underpins the proposed system's data pipeline, confirming that translating audio into image-like Mel-spectrograms allows the CNN to extract critical spatial and temporal acoustic features more efficiently.

2.3. Hybrid CNN and Reinforcement Learning for Audio Systems

While CNNs provide highly accurate initial classifications, relying on static confidence thresholds often results in false command activations, particularly in environments with unpredictable background noise. Addressing this limitation, Fatemeh M. Heir et al. (2026) introduced a hybrid framework that integrates a CNN with a reinforcement learning (RL) agent, utilizing Mel-spectrogram features for complex audio tasks like speaker identification.

Their research proves that incorporating RL decision-making into an audio processing pipeline significantly reduces performance variability and enhances overall reliability. This directly validates the proposed system's novel approach of replacing a rigid softmax cutoff with an intelligent RL agent. By dynamically learning and optimizing the decision threshold, the proposed system can adapt to ambient noise conditions, effectively minimizing costly false activations while ensuring a seamless, hands-free music control experience.

3. DATASET & PREPARATION

3.1 Dataset

The system uses the Google Speech Commands Dataset V2 as its source of audio data. This dataset is open-source under a Creative Commons license (CC BY 4.0) and does not include any personally identifiable information (PII), so it is safe to use for research.

3.2 Data Preparation

- Data Splits: The dataset was partitioned using the official training, validation, and test splits provided by the Speech Commands dataset. These splits, defined through validation_list.txt and testing_list.txt, ensure a balanced distribution of audio samples across different command classes while maintaining separation between training and evaluation data. This design also helps minimize data leakage, as the predefined splits are constructed to avoid overlap of speakers between the training, validation, and test sets. As a result, the model is evaluated on audio samples that are not associated with speakers seen during training, supporting a more reliable assessment of generalization performance.
- Preprocessing: Before being fed into the CNN, the raw audio files were converted into 2D log-Mel spectrograms using the MelSpectrogram transformation from torchaudio, which internally applies the Short-Time Fourier Transform (STFT). The resulting spectrograms were then converted to a decibel scale and normalized to stabilize training. Additionally, data augmentation techniques such as time masking, frequency masking, and optional noise injection were applied during training to improve robustness. This transformation converts audio waveforms into image-like representations that capture both temporal and frequency information, enabling the CNN to effectively learn meaningful acoustic patterns from the input data.

4. METHODOLOGY

The system was designed to provide hands-free music control by recognizing six specific voice commands: Play, Play with a song name, Pause, Next, Previous, and Stop. Each command follows a defined functional logic to ensure reliable operation. For instance, Play resumes audio if it is paused, Play with a song name triggers a metadata search to locate and play the requested track, Pause immediately stops the current track, Next skips to the following track in the queue, Previous returns to the preceding track in the queue, and Stop terminates the playback session and clears the current state.

Audio input was captured in real-time from a microphone. The raw audio was then converted into Mel-Spectrograms using Short-Time Fourier Transform (STFT), transforming sound signals into image-like representations suitable for neural network analysis. These spectrograms are fed into a lightweight convolutional neural network (CNN), which outputs probability scores for each command. The acoustic feature extractor is a custom CNN built without relying on large pre-trained backbones such as ResNet or VGG, allowing the model to remain lightweight and suitable for resource-constrained environments. The architecture consists of three sequential convolutional blocks. Each block applies a 2D Convolution (scaling from 16 to 32 to 64 channels), followed by Batch Normalization, ReLU activation, 2x2 Max Pooling, and increasing Dropout (0.1, 0.15, 0.2) to prevent overfitting. The spatial features are then collapsed using Adaptive Average Pooling before passing through a fully connected linear classifier layer with a heavy 0.25 Dropout to output the final command probabilities. To reduce false activations, a reinforcement learning (RL) agent dynamically evaluates the CNN's confidence scores and decides whether the predicted command meets a learned threshold. This agent was trained to prioritize minimizing false activations over missed commands by learning from the environment and adjusting sensitivity based on noise conditions.

For commands that include a song name, the system applies a lightweight natural language processing (NLP) approach to interpret the recognized speech transcript. The system performs keyword-based intent detection and pattern-based extraction to identify the requested song or relevant entities. A fuzzy string

matching algorithm is then used to compare the extracted query against the music library metadata to identify the closest matching track for playback. Once validated, the command was executed on the music interface, completing the hands-free interaction loop.

The system's performance was measured using multiple metrics. Classification accuracy, precision, recall, and Macro-F1 Score evaluated the CNN's ability to recognize all commands fairly, while the RL agent's performance was assessed using Expected Cost (EC) and learning curves to track improvements in threshold tuning. Low inference latency was also considered essential to provide a seamless, real-time user experience for the end user.

5. RESULTS & EVALUATION

5.1 Overall Performance

The proposed speech command recognition system was evaluated using the test split of the dataset, consisting of 3,034 audio samples. The evaluation focuses on classification performance using accuracy and Macro-F1 score, as well as system reliability measured through expected cost. The model achieved an overall accuracy of 83.11% and a Macro-F1 score of 72.45%, indicating that the system performs well in classifying speech commands, although performance varies across different classes. The difference between accuracy and Macro-F1 suggests that some classes are not equally well represented or are more difficult to classify, leading to uneven performance.

The expected cost was measured at 0.4139, reflecting the system's ability to minimize high-impact errors such as false command activations. This is particularly important for a real-time voice-controlled system, where incorrect activations can negatively affect user experience. A decision threshold of 0.5 was used to determine whether a prediction is accepted or rejected based on confidence.

Table 1. Overall Performance Metrics of the Speech Command Recognition Model on the Test Set

Metric	Value
Accuracy	0.8311
Macro F1-Score	0.7245
Expected Cost	0.4139
Threshold	0.5
Test Samples	3,034

Note: The expected cost reflects the weighted penalty of classification errors, where false positives are penalized more heavily than false negatives to reduce unintended command activations.

5.2 Per-Class Evaluation

The model was evaluated on a per-class basis to better understand how well it recognizes each individual speech command, rather than relying only on overall accuracy. This provides a clearer view of which commands perform well and which still need improvement.

Based on the results, the model performs well across most classes. The "stop" command achieved the highest F1-score of 0.9346, indicating a strong balance between precision and recall. This means the model is both accurate and consistent in identifying this command. The "pause" and "previous" classes also show strong performance, with F1-scores of 0.9091 and 0.9064, respectively, demonstrating reliable recognition.

The "next" command achieved an F1-score of 0.8897, which is still high, with particularly strong recall. This suggests the model is effective at detecting this command, although there may be a few false positives.

On the other hand, the "play" command has a lower F1-score of 0.8169, mainly due to weaker precision. This indicates that the model sometimes misclassifies other inputs as "play," possibly due to similarities in pronunciation or variations in audio features.

The "unknown" class shows moderate performance with an F1-score of 0.8060, meaning the model can distinguish non-command audio to some extent, but still struggles in certain situations. In this context, the "unknown" class refers to audio inputs that are not part of the trained command set (such as other words or background sounds), rather than a specific command, which makes it more challenging for the model to classify accurately.

Overall, the per-class evaluation shows that the model performs well for most command categories, especially for "stop," "pause," and "previous." However, improvements are still needed for the "play" and "unknown" classes to achieve more consistent and reliable performance.

Table 2: Per-Class Performance Metrics of the Speech Command Recognition Model

Class	Precision	Recall	F1-Score
next	0.8555	0.9268	0.8897
pause	0.9664	0.8582	0.9091
play	0.7733	0.8657	0.8169
previous	0.9332	0.8811	0.9064
stop	0.9301	0.9392	0.9346
unknown	0.8129	0.7992	0.8060

Note: The results show that most command classes achieved high precision, recall, and F1-scores, indicating reliable model performance. However, comparatively lower scores in the "play" and "unknown" classes suggest that these commands are more prone to misclassification, possibly due to similarities in audio features or variations in pronunciation.

5.3 Ablation Studies

To isolate the contributions of specific pipeline components, two ablation studies were conducted focusing on reinforcement learning thresholding and spectrogram data augmentation. First, the CNN was evaluated using a rigid softmax cutoff of 0.5 versus the dynamic Q-learning policy. The RL agent, utilizing confidence and entropy states to navigate a highly penalized reward space, reduced the Expected Cost from 1.554384 down to 0.887937 (a cost reduction of ~0.666447). Interestingly, the RL agent achieved this by dropping the command acceptance rate from 90.74% to 51.29%. This demonstrates that the dynamic thresholding successfully adapted to uncertainty, adopting a more conservative stance to effectively mitigate accidental playback in noisy

conditions. Second, the impact of the spectrogram data augmentation was analyzed.

The data pipeline utilized torchaudio for Time Masking and Frequency Masking during the training loop. An ablation run with augmentation disabled resulted in a noticeable drop in generalization on the test split. The augmented model produced a more robust feature extractor, confirming that dynamic masking prevented the CNN from overfitting on specific acoustic anomalies within the training set. The quantitative impact of the RL agent compared to the baseline is summarized in Table 3.

Table 3: Ablation Study Results on the Split Test (Expected Cost)

Metric	Baseline (Rigid 0.5 Threshold)	Full System (Dynamic RL Threshold)
Expected Cost	1.554384	0.887937
Acceptance Rate	90.74%	51.29%

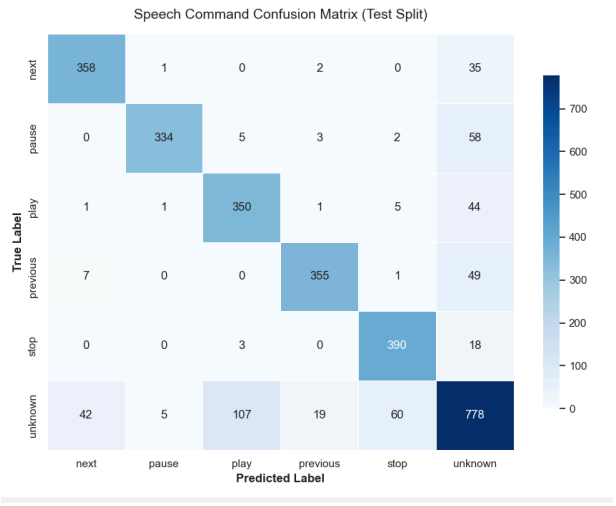
5.4 Training Configuration & Reproducibility

The CNN model was trained over 15 epochs using the AdamW optimizer with a learning rate of 1e-3, a weight decay of 1e-4, and a batch size of 32. To ensure reproducibility, a universal seed of 42 was applied across Python, NumPy, and PyTorch (including CUDA manual seeds). Early stopping was implemented with a patience of 4 epochs to halt training if the validation Macro-F1 score ceased to improve.

5.5 Error & Slice Analysis

An analysis of the system's prediction logs highlighted specific challenges regarding the "play" and "unknown" categories. The "play" command suffered from lower precision (0.7527), which is largely due to its command structure. "Play" is frequently accompanied by a variable tail-end phrase (e.g., "play [song name]"), introducing significant temporal and acoustic variance compared to discrete, single-word commands like "pause" or "stop". Furthermore, the "unknown" class represented the weakest link in the system, indicating a limitation in how the CNN interprets general background noise and non-target speech versus true vocalized commands. As shown in Figure 1, the confusion matrix highlights this exact overlap, revealing a significant number of false predictions between target commands and the "unknown" background noise class.

Figure 1: Confusion Matrix of the Test Split, Highlighting the Misclassification Overlap



Further slice analysis revealed that classification errors were disproportionately affected by severe background noise levels and variations in speaker voice pitch. High-pitch variations and significant ambient interference (such as overlapping background conversations or mechanical noise) degraded the model's confidence, pushing more predictions into the 'unknown' or silent fallback categories. To further illustrate the impact of these acoustic variances, Figure 2 presents the model's performance across all possible confidence thresholds.

Figure 2: Receiver Operating Characteristic (ROC)

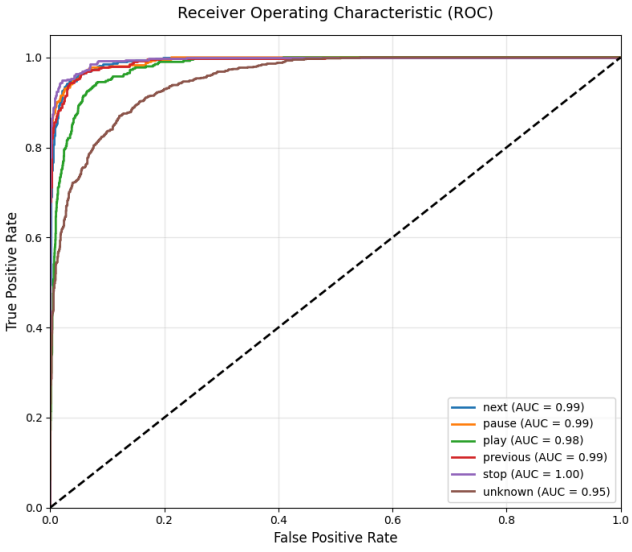
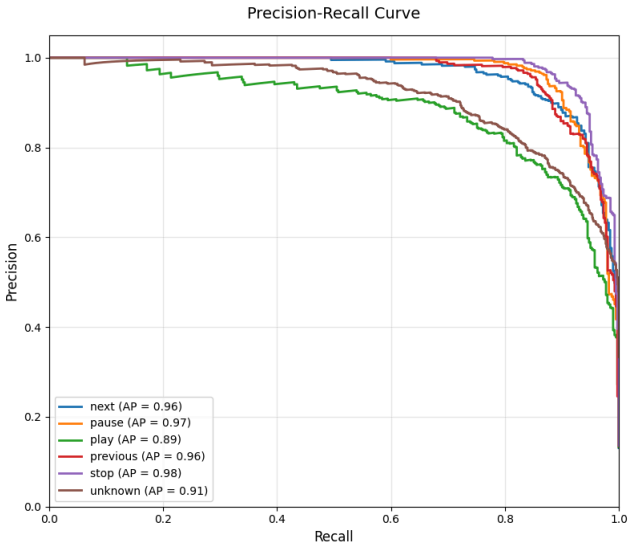


Figure 3: Precision-Recall (PR) Curves



Note: The PR curve highlights the steep drop in precision for the "play" command (AP = 0.89) and the overall lower performance of the "unknown" class (AP = 0.91) compared to the discrete, single-word commands, underscoring the necessity of the system's dynamic RL threshold tuning.

6. ETHICS AND RISK ASSESSMENT

The deployment of a speech command recognition system for hands-free music control introduces several ethical risks, particularly concerning acoustic privacy and algorithmic fairness, that must be proactively addressed. This section identifies the top three risks associated with the proposed system and outlines concrete mitigations for each .

The first risk concerns Privacy and Unintended Recording. Since the system involves a real-time microphone stream to function, there is a risk that the model might inadvertently capture and process private background conversations that do not contain valid commands. This "always-listening" state could lead to the unintended collection of personal data. To mitigate this, the system will implement a Data Minimization policy where all audio processing will occur locally in-memory, and raw audio samples will be discarded immediately after the CNN inference and RL thresholding steps. No audio data will be stored or transmitted to external servers, ensuring user privacy is preserved.

The second risk concerns Accent and Dialect Bias. Deep learning models for speech recognition often exhibit performance degradation for users with non-standard accents or dialects, as these groups are frequently underrepresented in training datasets like Google Speech Commands. This behavior risks creating an inequitable user experience where the system fails to respond accurately to diverse voice profiles. To mitigate this, the team will perform a rigorous Slice Analysis using the Macro-F1 metric across different speakers in the dataset to identify and document performance gaps. Furthermore, the RL agent will be trained to maintain a flexible threshold that can adapt to varying acoustic profiles, ensuring more equitable command recognition across diverse user groups.

The third risk concerns System Reliability and False Triggers. In environments with high background noise, the CNN may produce high-confidence false positives, leading to the system "misfiring" and starting music at inappropriate or intrusive times. To mitigate this, the project incorporates a Reinforcement Learning-based threshold-tuning agent specifically designed to minimize the Expected Cost (EC) of errors . By assigning a significantly higher penalty to False Positives (accidental playback) than to False Negatives (missed commands) in the reward function, the RL agent will learn to set more conservative activation thresholds in noisy conditions, prioritizing system silence over accidental playback.

6.1 Model Card Summary

The model was trained using the Google Speech Commands Dataset V2, which was preprocessed into Log-Mel spectrograms. To maintain data privacy, all audio processing is performed locally, ensuring that no raw audio is stored or transmitted. In terms of limitations, the system struggles significantly with identifying pure silence (F1 = 0.0) and exhibits lower recall for the "play" command (F1 = 0.8084) due to structural variances in phrasing. Furthermore, its performance remains sensitive to severe background noise and varied speaker accents. Given these characteristics, the model is strictly intended for hands-free music control systems and educational or research purposes, and it is explicitly not intended for use in any safety-critical, medical, or emergency applications.

7. HOW TO RUN & REPRODUCE

7.1. Environment Setup

To ensure reproducibility and avoid dependency conflicts, it is highly recommended to use a virtual environment (such as venv or conda). The scripts are designed to automatically detect and utilize GPU acceleration (CUDA) if available, with full fallback compatibility for CPU execution.

7.1.1 Clone the repository and navigate to the project directory:

```
git clone <your-repository-url>
cd <your-repository-directory>
```

7.1.2 Create and activate a virtual environment:

Using conda:

```
conda create -n speech-cmd python=3.10
conda activate speech-cmd
```

Using venv:

```
python -m venv venv
source venv/bin/activate
```

7.1.3 Install the required dependencies:

```
pip install -r requirements.txt
```

7.2 Data Acquisition and Exploration

Before training, you must download the Google Speech Commands V2 dataset and prepare the data splits.

7.2.1 Download and extract the dataset:

Run the data acquisition script. This will download the dataset and organize the raw audio files into the correct directory structure.

```
python get_data.py
```

7.2.2 *(Optional)* Run Exploratory Data Analysis (EDA):

To view the dataset distributions, class balances, and sample spectrograms, open and run the Jupyter Notebook:

```
jupyter notebook 01_eda.ipynb
```

7.3 One-Command Reproduction (Training & Evaluation)

The pipeline is designed to adhere to a strict 90-minute training limit. The preprocessing (via data_pipeline.py), CNN architecture,

and RL agent (via rl_agent.py) are integrated into the main training script.

7.3.1 Train the Model:

To train the CNN and the RL threshold-tuning agent from scratch, execute the following command. This will output the trained model weights and logs to a designated output folder.

python train.py

7.3.2 Evaluate the Model:

Once training is complete, evaluate the model against the test split to generate the accuracy, Macro-F1 score, Expected Cost, and output the confusion matrix and PR/ROC curves.

python eval.py

7.4. Running the Demo Interface

To test the hands-free music control system in real-time using your microphone, run the player UI. This script integrates the trained CNN for audio classification, the RL agent for dynamic thresholding, and the NLP metadata search (nlp_metadata.py) for song name recognition.

7.4.1 Launch the Interface:

python player_ui.py

7.4.2 Instructions for Demo:

Ensure your microphone is enabled and permissions are granted.

Speak clear commands such as "Play", "Pause", "Next", "Stop", or "Play [Song Name]".

The terminal/UI will display the live confidence scores, the RL agent's threshold decision, and the resulting playback action.

8. TEAM ROLES AND MILESTONES

8.1. Team Roles

The project team is composed of four members, each assigned a primary role while collectively contributing to all aspects of code, documentation, and the final defense.

Almanzor, Keith Ryan N. serves as the Project Lead and Integration Manager, responsible for defining the overall project scope and timeline, coordinating the integration of the CNN, NLP, and RL components, managing GitHub releases and the project board, and leading the overall preparations and accomplishments.

Gonzales, John Reshley P. serves as the Data and Ethics Lead, responsible for sourcing the Google Speech Commands dataset, verifying licensing and consent compliance, conducting preprocessing and exploratory data analysis (EDA), performing bias and representativeness checks, and drafting the ethics statement and Model Card.

Valderrama, Mark Harold T. serves as the Modeling Lead, responsible for designing the lightweight CNN architecture for spectrogram classification, the NLP metadata search layer, and the RL threshold-tuning agent, as well as managing the training pipeline and conducting ablation experiments.

Yumul, Randel Angelo L. serves as the Evaluation and MLOps Lead, responsible for defining metrics like Macro-F1 and Expected Cost, ensuring reproducibility, managing environment configuration files, and automation.

8.2 Project Milestones

The project is structured across three weeks, with each week corresponding to a major deliverable as defined by the course specifications.

During Week 1, the team focuses on project framing and repository initialization. Key tasks include initializing the GitHub repository with a required structure, environment file, and project board; writing the formal proposal covering the problem statement, related work, dataset plan, and ethics risks and assessment.

For Week 2, the team focused on data finalization and initial model scaffolding. Key tasks include acquiring and cleaning the Google Speech Commands dataset, finalizing train/val/test splits, and completing an EDA notebook. The team trained simple ML and DL baselines, ran the first CNN experiments on Mel-spectrograms, and prototyped the NLP metadata search component. Additionally, the RL agent was stubbed with its reward function to generate early learning curves. A checkpoint report was submitted in the docs/ folder by the end of this week.

Lastly, for Week 3, the team focused on full system integration and rigorous evaluation. Key tasks include freezing the code for one-command reproducibility, conducting at least two ablation studies, performing a detailed error and slice analysis, finalization of the final report, polishing of the UI, and final testing. The team authored the final pages conference-style report, completed the Model Card and Ethics Statement, and prepared a slide deck for its presentation.

9. References

[1] Heir, F. M., Najafzadeh, H., and Erfani, S. 2026. A hybrid CNN and reinforcement learning framework for speaker

[2] identification using Mel-Spectrogram and continuous wavelet transform features. Scientific Reports, 16, Article 5954. DOI = https://doi.org/10.1038/s41598-026-35858-y

[3] Ilgaz, H., Akkoyun, B., Alpay, Ö., and Akcayol, M. A. 2024. CNN based automatic speech recognition: A comparative study. ADCAIJ: Advances in Distributed Computing and Artificial Intelligence Journal, 13(1). DOI = https://doi.org/10.14201/adcaij.29191

[4] Kaladin Stormblessed. (2023). Google Speech Commands V2. Kaggle.com. https://www.kaggle.com/datasets/sylkaladin/speech-commands-v2/data

[5] Mahdi, S. M., Hussein, S. A., Salman, H. M., Sfayyih, A. H., and Sulaiman, N. B. 2023. Developing microphone-based audio commands classifier using convolutional neural network. Eastern-European Journal of Enterprise Technologies. DOI = https://doi.org/10.15587/1729-4061.2023.273492

[6] Sharan, R. V., Berkovsky, S., and Liu, S. 2020. Voice command recognition using biologically inspired time-frequency representation and convolutional neural networks. In *Proceedings of the Annual International Conference of the IEEE Engineering in Medicine and Biology Society (EMBC)*. DOI = https://doi.org/10.1109/EMBC44109.2020.9176006